Types of Pointcut :-

1:- **Execution** :- matches a particular method in a particular class.

Method return Type.

Class name with method Parameter.

```
@Before("execution(public String com.learning.Employee.fetchEmployee())")
```

optional

→ (*) Wildcard : matches any single item.

Qu1: Matches any return type.

```
@Before("execution(* com.learing.Employee.fetchEmployee())")
Public void beforeMethod(){
    S.O.P @("inside before Method");
}
```

Qu2: Matches any method with single parameter String

```
@Before("execution(* com.learning.Employee.*(String))")
```

Qu3: Matches fetchEmployee method that take any single parameter

```
@Before("execution(String com.learning.Employee.fetchEmployee(*))")
```

→ (..) Wildcard : matches 0 or more than

Qu1: Matches fetchEmployee method that take 0 or More parameters.

```
@Before("execution(String com.learning.Employee.fetchEmployee(..))")
```

Qu2:- Matches fetchEmployee method in 'com.learning' package and subpackage classes.

```
@Before("execution(String com.learing.. fetchEmployee())")
```

Concept && Coding

43: Match any method in 'com.learning' package and subclasses.

~~@Before (String~~

```
@Before("execution(String com.learning..*())")
```

~~2: Within : matches all method within any class or package.~~
~~* @within is class level.~~

```
@RestController
@RequestMapping(value = "/api")
Public class Employee {
    @Autowired
    EmployeeUtil employeeUtil;

    @GetMapping
    Public String fetchEmployee(){
        employeeUtil.employeeHelperMethod();
        return "item fetched";
    }
}
```

3 @within : matches any method in a class which has this annotation.

```
@Service
Public class EmployeeUtil {
    public void employeeHelperMethod(){
        S.o.p("employee helper method called");
    }
}
```

```
@Aspect
@Component
Public class LoggingAspect {
    @Before("@within(org.springframework.stereotype.Service)")
    Public void beforeMethod(){
        S.o.p("inside before aspect");
    }
}
```

Concept && Coding

Cracking The Coding interview

Chapter 1 :

Qu 1.1

2 : Within : matches all method within any class or package.

→ This pointcut will run for each method in the class Employee

```
@Before("within(com.learning.Employee)")
```

→ This pointcut will run for each method in the package and subpackage.

```
@Before("within(com.learning..*)")
```

4 : @annotation :- matches any method that is annotated with given annotation.

```
@RestController
@RequestMapping(value = "/api/")
Public class Employee{

    @GetMapp(path = "/fetchEmployee")
    Public String fetchEmployee(){
        return "item fetched";
    }
}
```

```
@Component
@Aspect
Public class LoggingAspect{
    @Before("@annotation(org.springframework.web.bind.GetMapping)")
    Public void beforeMethod(){
        s.o.p("Aspect");
    }
```

Concept && Coding

5. Args :- matches any method with particular arguments (or parameters)

```
@Before("args(String,int)")
```

* If Instead of primitive type, we need object, then we can give like this.

```
@Before("args(com.concept.learning.Employee)")
```

6: @args :- matches any method with particular parameters and that parameter class is annotated with particular annotation.

```
@Before("@args(org.springframework.stereotype.Service)")
```

7: target :- matches any method on a particular instance of class

```
@Before("target(com.learning.EmployeeUtil)")
```

```
@Before("target(com.learning.IEmployee)")
```

Interface.

* Combining two pointcuts using.

1. && (boolean and)
   || (boolean or)

Concept && Coding

Named Pointcuts

```
@Aspect
@Component
Public class LoggingAspect{

  @Pointcut("execution(* com.learning.EmployeeController.*())")
  Public void customPointcutName(){
     // always stay empty
  }

  @Before("customPointcutName()")
  Public void beforeMethod(){
        S.o.p("inside beforeMethod aspect");
  }
}
```

Advice:-

Its an action, which is taken @Before or @After or @Around the method execution.

@Around

As the name says, it surrounds the method execution (before and after both).

```
@Aspect
@Component
Public class LogginAspect
  @Aroud("execution(* com.learning.EmployeeUtill.*())")
  Public void aroundMethod(ProceedJoinPoint joinPoint) throw Throwab
     S.o.p("inside before Method aspect");
     joinPoint.proceed();
     S.o.p("inside after Method aspect");
  }
}
```

Concept && Coding

By Now, few questions we all should have:

1:- How this interception works?

2:- What if we have 1000s of pointcut, so ~~what~~ whenever I invoke a method, does

Join Point:- It generally considered a point, where actual method invocation happens.

How to AOP internally work??

1. When application startup happens

1. → look for @Aspect annotation classes.
2. → Parse the Pointcut Expression
   - → Done by PointcutPaser.java class

   It's simplyfy those expressi Save in a particular data structure. So that is easy to do a matching

   → Store in a efficient data structure or cache after parsing
3. → look for @Component, @Service, @Controller etc. annotation classes.
4. → For each class, it check if its eligible for interceptio based on pointcut Expression.
   - → Done by AbstractAutoProxyCreater.java clas
5. → If yes, it creates a Proxy using JDK Dynamic proxy or CGLIB porxy. This proxy class has code which execute advice [illegible] method execution happens

Concept&Coding

and after than advice if any.

Code Generation Library
proxy

Concept && Coding